

Байгууллагын бүтээгдэхүүн хүргэлтийн нэгдсэн систем

Компьютерын ухааны тэнхим

Оюутан: s21c011b, Ү. Тамир

Удирдагч багш: Н.Соронзонболд

1. Удиртгал

Монгол Улсын жижиг, дунд бизнесийн байгууллагууд (ЖДБ) хоорондоо бараа захиалах, хүргэх үйл явцаа одоо хүртэл голдуу утас, мессеж, цаасан бичгээр гүйцэтгэсээр байна. Одоогийн практикт (i) жолооч бүртэй тусад нь утсаар ярьж захиалга бүрт 10–15 минут зарцуулдаг, (ii) хүргэлтийн баримтыг гар бичмэлээр хөтлөх нь андуурал, маргааныг нэмэгдүүлдэг, (iii) байгууллага бүр өөрийн цалинжуулдаг хүргэлтийн багтай байхыг шаарддаг зэрэг асуудал байсаар байна. Ийм гараар зохицуулдаг хэлбэр нь цаг алдагдуулж, зохион байгуулалтын алдаа үүсгэж, явцыг хянахад төвөгтэй болгодог. Монголын зах зээлд B2B (business-to-business буюу байгууллага хоорондын) хүргэлтийн нэгдсэн шийдэл хараахан төлөвшөөгүй бөгөөд одоогийн тоглогчид голдуу dispatcher (захиалга хуваарилагч ажилтан)-т тулгуурласан эсвэл bid-based (үнийн саналын уралдаан) нэг талтай загвартай хэвээр байна.

Operator-гүй загвар. “Operator-гүй” (оператор-free) гэдэг нь систем дотор захиалгыг гараар хуваарилдаг төвлөрсөн ажилтан (диспетчер) байхгүй, харин хүргэгчид (courier) боломжит даалгаврын жагсаалтаас өөрсдөө сонгон авдаг (self-service claim) загварыг хэлнэ. Энэ нь хүний хүчин зүйлийн алдаа, bottleneck-ийг (хүлцэгдэх хүзүү) арилгана.

Системийн оролцогчид. Платформ дөрвөн үндсэн оролцогчтой: (1) *Байгууллага* (organization/tenant) — бүтээгдэхүүн бүртгэж, захиалга хүлээн авч, хүргэлт зарладаг; (2) *Хэрэглэгч* (customer) — захиалга өгч, ePOD OTP-ээр хүлээн авсан баталгаажуулалт хийдэг; (3) *Хүргэгч* (courier) — даалгаврыг claim (барьж авах) хийж, хүргэлтийг гүйцэтгэдэг; (4) *Админ* (admin) —

байгууллагын бүртгэл болон системийн ерөнхий тохиргоог хянадаг.

Судалгааны зорилго нь:

- Operator-гүй (төвлөрсөн зохицуулагчгүй), multi-tenant (олон байгууллагыг нэг системд тусгаарлан үйлчлэх) B2B хүргэлтийн платформыг Supabase (PostgreSQL + PostgREST + RLS буюу мөрийн түвшний аюулгүй байдал) технологи дээр боловсруулах,
- Concurrency (зэрэгцээ хандалт) хамгаалалтыг database (өгөгдлийн сан) түвшинд шийдэж баталгаажуулах,
- Multi-tenant data isolation-ыг (байгууллага хоорондын өгөгдлийн тусгаарлалтыг) нотлох,
- ePOD (electronic Proof of Delivery буюу цахим хүргэлтийн нотолгоо) OTP (One-Time Password, нэг удаагийн нууц үг) верификацыг аюулгүй болгох,
- Туршилтаар системийн performance-ийг (ажиллагааны хурд) үнэлэх.

Судалгааны асуултууд (RQ — Research Question) дараах гурван чиглэлд төвлөрсөн: RQ1 — FCFS (First-Come-First-Served, эхэлж ирсэн нь эхэлж үйлчлүүлэх) claim загварын concurrency correctness (зэрэгцээ хандалтын зөв эсэх); RQ2 — RLS-ийн multi-tenant isolation найдвартай байдал; RQ3 — BaaS-mediated (Backend-as-a-Service буюу backend-ийг үйлчилгээ байдлаар хүргэх платформуор зуучилсан) архитектурын production viability (үйлдвэрлэлд ашиглахад тэнцэх байдал). Эдгээрийг concurrency test (зэрэгцээ хандалтын сорил), load test (ачааллын сорил), security penetration test (аюулгүй

байдлын нэвтрэлтийн сорил), functional E2E (end-to-end буюу эхнээс эцэс хүртэл) test гэсэн 4 төрлийн туршилтаар хэмжив.

2. Судалгааны сэдвийн онол, өнөөгийн байдал

B2B хүргэлтийн платформын онолын суурь нь таван үндсэн чиглэлд тулгуурласан: last-mile (сүүлчийн миль буюу эцсийн хүргэлтийн үе) логистик, two-sided marketplace (хоёр талт зах зээл), multi-tenant isolation, concurrency control (зэрэгцээ хандалтын хяналт), BaaS (Backend-as-a-Service) архитектур.

Last-mile хүргэлт нь нийт тээврийн зардлын 13–75%-ийг эзэлдэг тул crowdsourced (олон нийтийн оролцоотой) загвар нь уламжлалт өөрийн паркттай системийг орлох шинэ чиг хандлага болсон. Two-sided marketplace онолоор (Rochet & Tirole 2003) B2B платформын гол саад нь liquidity (хоёр талын оролцогчдын хүрэлцэх нягтрал) тул cold-start (хэрэглэгч цөөн эхний үе) шатанд FCFS загвар давуу талтай.

Multi-tenant өгөгдлийн тусгаарлалтыг Chong (2006), Bezemer & Zaidman (2010) нар database түвшинд хэрэгжүүлэхийг зөвлөсөн. Үүнийг PostgreSQL-ийн Row Level Security (RLS, мөрийн түвшний хандалтын хяналт) нь хүснэгтийн мөр бүрд USING/WITH CHECK policy (бодлого) хавсарган хангана.

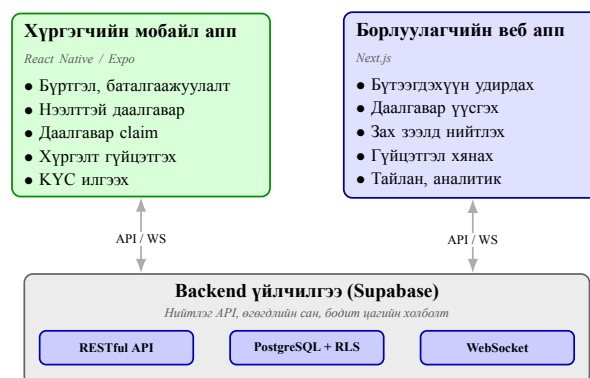
Concurrency control-ыг Bernstein (1987), Kleppmann (2017) нар pessimistic (урьдчилан түгжих) / optimistic (сүүлд шалгах) хоёр хандлагад ангилсан. Write-heavy (бичих үйлдэл давамгайлсан) workload-д pessimistic илүү найдвартай тул PostgreSQL-ийн SELECT ... FOR UPDATE SKIP LOCKED (түгжигдсэн мөрийг алгасах SQL хэлбэр) нь race condition-ыг (зэрэгцээ мөргөлдөөнийг) 0 магадлалаар шийднэ.

Харьцуулсан судалгаанд Uber Freight (ML буюу машин сургалтад суурилсан enterprise (томоохон корпорацийн) платформ) болон Papa Logistics (bid-based буюу илгээгч захиалга нийтлээд жолооч нар үнийн санал илгээж, илгээгч харьцуулан сонгодог Монголын платформ) хоёрыг авч үзсэн. Эдгээрээс dispatcher bottleneck

(захиалга хуваарилагчид төвлөрсөн хүлцэгдэх хүзүү), data isolation сул, audit trail (үйлдлийн түүхийн бүртгэл) хязгаарлагдмал, API (Application Programming Interface, програмчлалын интерфэйс) integration сул гэсэн 4 архитектурын зөрүүг тодорхойлсон.

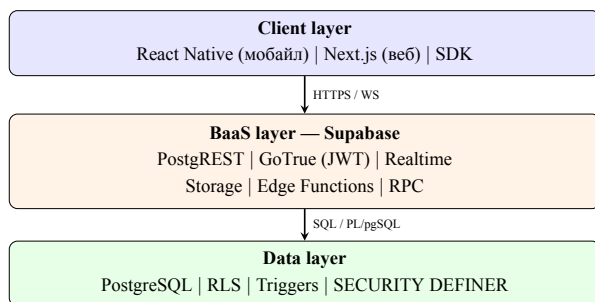
3. Судалгааны арга зүй

Судалгааг Design Science Research (загварчлалын шинжлэх ухааны судалгаа) арга барилаар 3 үе шатаар (загварчлал → хөгжүүлэлт → туршилт) Agile (уян хатан, богино давталттай хөгжүүлэлтийн арга) давталттай хэрэгжүүлэв. Платформ нь хоёр бие даасан клиент апп (Хүргэгчийн React Native мобайл апп, Борлуулагчийн Next.js веб апп) болон тэдгээртэй HTTPS/WebSocket-ээр холбогдсон нэгдсэн Supabase backend-ээс бүрдэнэ (Зураг 1).



Зураг 1: Системийн ерөнхий бүтэц: 2 клиент апп + нэгдсэн backend

Архитектурын гол онцлог нь **BaaS-mediated architecture** (BaaS-ээр зуучилсан архитектур) бөгөөд уламжлалт three-tier (гурван давхарга бүхий сонгодог client-server-database) биш, Supabase BaaS платформоор бүх backend (серверийн тал) үйлчилгээг зуучилсан гурван давхаргат бүтэц юм (Зураг 2).



Зураг 2: BaaS-mediated архитектурын гурван давхаргат бүтэц

Client layer (клиент/хэрэглэгчийн давхарга) нь хэрэглэгчийн интерфэйсийг хариуцна. BaaS layer нь Supabase платформоор API-г автоматаар үүсгэж, JWT (JSON Web Token — нууцлан гарын үсэг зурсан хандалтын токен) authentication (баталгаажуулалт), realtime (шууд дамжих) Web-Socket, storage (файл хадгалалт) болон RPC (Remote Procedure Call — алсын функц дуудлага) дуудлагыг нэгтгэнэ. Data layer нь PostgreSQL + RLS + триггер (автомат ажилладаг database функц) + SECURITY DEFINER (өндөр эрхээр гүйцэтгэгддэг) функцээр бизнесийн логикийг database дотор хэрэгжүүлнэ.

Өгөгдлийн түвшинд 11 хүснэгт бүхий schema (өгөгдлийн бүтэц) зохион байгуулж, multi-tenant isolation-ыг org_id талбар дээр суурилсан RLS-ээр, зэрэгцээ claim-ийг FOR UPDATE SKIP LOCKED lock-оор (түгжих механизм) хангасан. Бизнесийн логикийн гол функцүүд: claim_delivery_task (атомик — хуваагдашгүй байдлаар даалгавар оноох), verify_epod_otp (bcrypt — аюулгүй нууц үг хэш хийх алгоритмаар OTP баталгаажуулах), мөн автомат төлөв шилжилт, орлого бүртгэлийн триггерүүд.

Proof of Delivery (ePOD — хүргэлтийн баталгаа)-г 6 оронтой OTP код (bcrypt hash — нууцалсан хэш, 10 мин expiry буюу хүчинтэй хугацаа, 5 оролдлогын хязгаар) хэлбэрээр хэрэгжүүлсэн. Client-server contract-ыг (клиент–сервер хоорондын өгөгдлийн гэрээ) хамгаалах schema drift prevention (схемийн зөрчил үүсэхээс сэргийлэх) стратегийг Supabase CLI (командын мөрийн хэрэгсэл)-ийн auto-generated (автоматаар үүсгэсэн) TypeScript

төрөл + CI (Continuous Integration — тасралтгүй нэгдэл) шатны харьцуулалтаар хангасан.

Системийн үндсэн урсгал. Бүртгэлээс орлого бүртгэл хүртэлх үйл явцыг 8 алхамт дараалалд зохион байгуулсан (Зураг 4). Энэ урсгал нь веб апп (1–4 алхам), мобайл апп (5–7 алхам), database триггер (8 алхам) гэсэн 3 бүрэлдэхүүнийг нэг гинжид уялдуулсан.

Өгөгдлийн урсгалын контекст (DFD Level 0).

Платформ нь 4 гадаад оролцогчтой харилцдаг (Зураг 3): борлуулагч бүтээгдэхүүн/даалгавар илгээж, гүйцэтгэл–орлогын тайлан хүлээж авна; хүргэгч нээлттэй даалгаврыг харж, KYC ба ePOD илгээнэ; имэйл үйлчилгээ OTP/мэдэгдэл дамжуулна; админ KYC-г батлан, бодлого тохируулна.



Зураг 3: DFD Level 0 — платформын контекст



Зураг 4: Системийн үндсэн урсгал: 8 алхамт үе шат

Аюулгүй байдлын олон давхаргат хамгаалалт. Системд 5 түвшний security (аюулгүй байдлын) давхарга хэрэгжсэн: (1) HTTPS/TLS (сүлжээн дэх шифрлэгдсэн холболтын протокол) сүлжээний шифрлэлт, (2) Supabase Auth + JWT, (3) Next.js middleware (хүсэлт хүлээн авах завсрын давхарга) / Nav-

igation guard (хуудсын хандалтыг шалгагч), (4) database-level RLS бодлого (org_id, courier_id, auth.uid()), (5) SECURITY DEFINER RPC функц дэх бизнесийн дүрмийн шалгалт.

Хүснэгт 1: Технологийн стекийн хураангуй

Давхарга	Технологи
Мобайл апп	React Native + Expo 54, TS 5.9
Веб апп	Next.js 16.1, React 19.2
Backend	Supabase (PostgreSQL 15+)
Security	RLS, JWT, bcrypt ePOD OTP

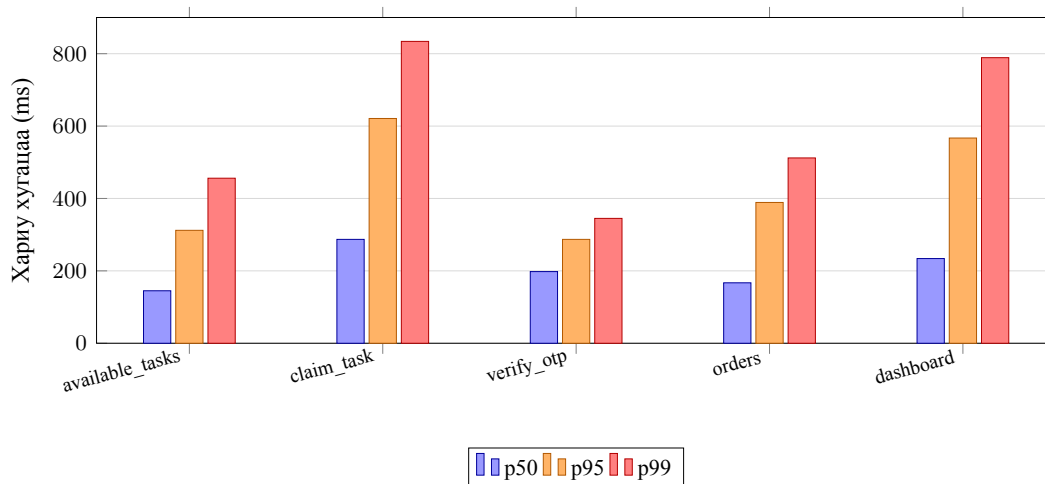
Туршилтын орчин: Supabase Pro tier (төлбөртэй үндсэн багц), PostgreSQL 15.6, PgBouncer transaction pooling (холболтын нөөцлөлт). Өгөгдлийн синтетик (хиймэл үүсгэсэн) багц: 20 байгууллага, 200 хэрэглэгч, 1,000 захиалга, 1,000 даалгавар. Ашигласан хэрэгсэл: @supabase/supabase-js (JavaScript client сан),

k6 0.54 (ачааллын сорилын хэрэгсэл), psql + SET request.jwt.claims (JWT claim-ийг шууд тохируулах SQL команд).

4. Судалгааны үр дүн, нэгтгэсэн дүгнэлт

Concurrency test. $N \in \{10, 50, 100\}$ хүргэгч нэг даалгаврыг зэрэг claim (барьж авах) хийх 30 сорилын туршид давхардсан claim нэг ч удаа гараагүй (100% амжилт). Дундаж хариу хугацаа $N=10$ -д 124 ms, $N=50$ -д 287 ms, $N=100$ -д 512 ms болсон нь $O(n)$ шугаман хэмжээтэй, deadlock (харилцан түгжрэл)-гүй pessimistic locking-ийн онолын хүлээгдсэн зан төлөвтэй нийцнэ.

Load test. k6-аар 100 VU (Virtual User — виртуал хэрэглэгч), 30 мин ачаалал өгөхөд бүх endpoint-ийн (API холбох цэг) p95 latency (95%-ийн хариу хугацаа) 2 секундын зорилтот хязгаарт багтсан:



Зураг 5: k6 load test — endpoint latency хуваарилалт (100 VU, 30 мин)

Security penetration test. Org A хэрэглэгчийн JWT-ээр Org B-гийн өгөгдөлд хандах 6 хувилбар (cross-tenant буюу байгууллага хооронд SELECT/UPDATE, SQL injection — хорлонтой SQL оруулах, pending courier access — зөвшөөрөгдөөгүй хүргэгчийн хандалт, JWT forgery — токен хуурамчлах) бүгд амжилтгүй болсон (6/6). OTP brute-force (дараалан оролдох халдлага) туршилтад 5 дахь алдаатай оролдлогын дараа сесс блоклогдож, brute-force амжилтын онолын магадлал $5 \times 10^{-40}\%$ болж буурсан.

Functional E2E. Байгууллагын бүртгэл → бүтээгдэхүүн нэмэх → захиалга → даалгавар нийтлэх → хүргэгч claim → pickup (авалт) → delivery (хүргэлт) → ePOD OTP → орлого бүртгэл гэсэн 8 алхамт урсгал зөв дарааллаар биелэв. Timestamp invariant (цагийн тэмдгийн өөрчлөгдөөгүй дараалал — assigned_at < picked_up_at < delivered_at < completed_at) зөрөлгүй, хүргэлтийн орлого автомат бүртгэгдсэн.

Хүснэгт 2: Харьцуулсан үнэлгээний хураангуй

Платформ	Чадамжийн хувь
Uber Freight	~35–40%
Papa Logistics	~70–75%

Хязгаарлалт. Туршилт нь Supabase Pro tier-ийн нөөцийн хязгаарт, 100 VU хүртэлх concurrent (зэрэгцээ) ачаалалд, synthetic (хиймэл үүсгэсэн) өгөгдөл дээр, single-region (нэг бүсийн сервер) тохиргоонд явагдсан. 500+ VU, multi-region (олон бүсийн), 3G/4G сүлжээний нөхцөл үнэлэгдээгүй. Бодит GPS tracking (байршил хяналт), QPay/SocialPay төлбөрийн integration (холболт), ML recommendation (машин сургалтаар санал болгох систем) нь future work (ирээдүйн судалгаа)-г үлдсэн.

Ерөнхий дүгнэлт. Судалгааны гол дүгнэлт нь B2B хүргэлтийн платформын найдвартай байдлыг database түвшинд хэрэгжүүлсэн concurrency control (FOR UPDATE SKIP LOCKED) ба RLS гэсэн хоёр тулгуурт механизмд суурилан бүтээх боломжтойг туршилтын тоогоор нотолсон явдал юм. FCFS claim загвар, row-level isolation, bcrypt-д суурилсан ePOD OTP гэсэн гурван архитектурын шийдэл хамтдаа ажилласнаар multi-tenant орчинд race condition, cross-tenant access, OTP brute-force гэсэн гурван гол эрсдэлийг бодитоор шийдсэн.

5. Ном зүй

Судалгааны гол эх сурвалжууд:

1. Chong, F., Carraro, G., & Wolter, R. (2006). Multi-tenant data architecture.
2. Bezemer, C.-P., & Zaidman, A. (2010). Multi-tenant SaaS applications: Maintenance dream or nightmare?
3. Ferraiolo, D. F., & Kuhn, D. R. (1992). Role-based access controls.
4. Bernstein, P. A., Hadzilacos, V., & Goodman, N. (1987). Concurrency Control and Recovery in Database Systems.
5. Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly.
6. Provos, N., & Mazières, D. (1999). A future-adaptable password scheme (bcrypt).
7. Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (RFC 7519).

8. Baldini, I., et al. (2017). Serverless computing: Current trends and open problems.
9. Rochet, J.-C., & Tirole, J. (2003). Platform competition in two-sided markets.
10. Alnaggar, A., et al. (2021). Crowdsourced delivery: A review.